

## **Unit 06: File Systems**

### **CONTENTS**

Objectives

Introduction

6.1 Obtaining User and System Information

6.2 Communicating with Other Users

6.3 The Filesystem

6.4 Pathnames

6.5 Working with Directories

6.6 Linux Access Permissions

6.7 Access Control Lists

6.8 Links

Summary

Keywords

Self Assessment

Answers for Self Assessment

Review Questions

Further Readings

### **Objectives**

After studying this unit, you will be able to:

- Obtain user and system information
- Communicate with other users
- Understand the filesystem
- Understand the pathnames
- Work with directories
- Understand the access permission
- Understand the Access Control Lists and Links

### **Introduction**

There are some utilities that provide information about who is using the system, what those users are doing, and how the system is running. To find out who is using the local system, you can employ one of several utilities that vary in the details they provide and the options they support.

#### **6.1 Obtaining User and System Information**

- who
- finger
- w

### Linux and Shell Scripting

---

The oldest utility, `who`, produces a list of users who are logged in on the local system, the device each person is using, and the time each person logged in. The `w` and `finger` utilities show more detail, such as each user's full name and the command line each user is running. You can use the `finger` utility to retrieve information about users on remote systems if the local system is attached to a network.

#### **who: Lists Users on the System**

The `who` utility displays a list of users who are logged in on the local system.

```
$ who
sam    tty4      2009-07-25 17:18
max    tty2      2009-07-25 16:42
zach   tty1      2009-07-25 16:39
max    pts/4     2009-07-25 17:27 (coffee)
```

The information that `who` displays is useful when you want to communicate with a user on the local system. If you need to find out which terminal you are using or what time you logged in, you can use the command `who am i`:

```
$ who am i
max    tty2      2009-07-25 16:42
```

#### **finger: Lists Users on the System**

You can use `finger` to display a list of users who are logged in on the local system. In addition to usernames, `finger` supplies each user's full name along with information about which device the user's terminal is connected to, how recently the user typed something on the keyboard, when the user logged in, and available contact information. If the user has logged in over the network, the name of the remote system is shown as the user's location.

```
$ finger
Login   Name           Tty      Idle  Login Time  Office ...
max     Max Wild      *tty2    3     Jul 25 16:42
max     Max Wild      pts/4    3     Jul 25 17:27 (coffee)
sam     Sam the Great *tty4    29    Jul 25 17:18
zach    Zach Brill    *tty1    1:07  Jul 25 16:39
```

On systems where security is a concern, the system administrator may disable `finger`. You can also use `finger` to learn more about an individual by specifying a username on the command line.

```
$ finger max
Login: max                      Name: Max Wild
Directory: /home/max           Shell: /bin/tcsh
On since Fri Jul 25 16:42 (PDT) on tty2 (messages off)
On since Fri Jul 25 17:27 (PDT) on pts/4 from coffee
    3 minutes 7 seconds idle
New mail received Sat Jul 25 17:16 2009 (PDT)
    Unread since Sat Jul 25 16:44 2009 (PDT)
Plan:
I will be at a conference in Hawaii all next week.
If you need to see me, contact Zach Brill, x1693.
```

You can also use `finger` to display a user's username. The `finger` utility, which is not case sensitive, can search for information on Helen using her first or last name.

```
$ finger HELEN
Login: hls                      Name: Helen Simpson.
...
$ finger simpson
Login: hls                      Name: Helen Simpson.
...
```

### w: Lists Users on the System

The `w` utility displays a list of the users who are logged in on the local system. The information that `w` displays is useful when you want to communicate with someone at your installation.

```
$ w
17:47:35 up 1 day, 8:10, 4 users, load average: 0.34, 0.23, 0.26
USER      TTY      FROM          LOGIN@   IDLE   JCPU   PCPU   WHAT
sam       tty4      -             17:18    29:14m 0.20s   0.00s   vi memo
max       tty2      -             16:42    0.00s   0.20s   0.07s   w
zach      tty1      -             16:39    1:07    0.05s   0.00s   run_bdgt
max       pts/4     coffee        17:27    3:10m   0.24s   0.24s   -bash
```

The first line the `w` utility displays includes the time of day, the period of time the computer has been running (in days, hours, and minutes), the number of users logged in, and the load average (how busy the system is).

## 6.2 Communicating with Other Users

### write: Sends a Message

These are used to exchange messages and files with other users either interactively or through email. It is enabled as a two-way communication.

The syntax of a write command line is: `write username [terminal]`. The username is the username of the user you want to communicate with. The terminal is an optional device name that is useful if the user is logged in more than once. You can display the usernames and device names of all users who are logged in on the local system by using `who`, `w`, or `finger`. To establish two-way communication with another user, you and the other user must each execute `write`, specifying the other's username as the username. The `write` utility then copies text, line by line, from one keyboard/display to the other. Sometimes it helps to establish a convention, such as typing `o` (for

### Linux and Shell Scripting

“over”) when you are ready for the other person to type and typing oo (for “over and out”) when you are ready to end the conversation. When you want to stop communicating with the other user, press CONTROL-D at the beginning of a line. Pressing CONTROL-D tells write to quit, displays EOF (end of file) on the other user’s terminal, and returns you to the shell. The other user must do the same.

If the Message from banner appears on your screen and obscures something you are working on, press CONTROL- L or CONTROL- R to refresh the screen and remove the banner. Then you can clean up, exit from your work, and respond to the person who is writing to you. You have to remember who is writing to you, however, because the banner will no longer appear on the screen.

### **mesg: Denies or Accepts Messages**

By default, messages to your screen are blocked. Give the following mesg command to allow other users to send you messages: `$ mesg y`

If Max had not given this command before Zach tried to send him a message, Zach might have seen the following message: **\$ write max**

write: max has messages disabled

You can block messages by entering `mesg n`. Give the command `mesg` by itself to display `y` (for “yes, messages are allowed”) or `n` (for “no, messages are not allowed”). If you have messages blocked and you write to another user, `write` displays the following message because, even if you are allowed to write to another user, the user will not be able to respond to you:

\$ write max

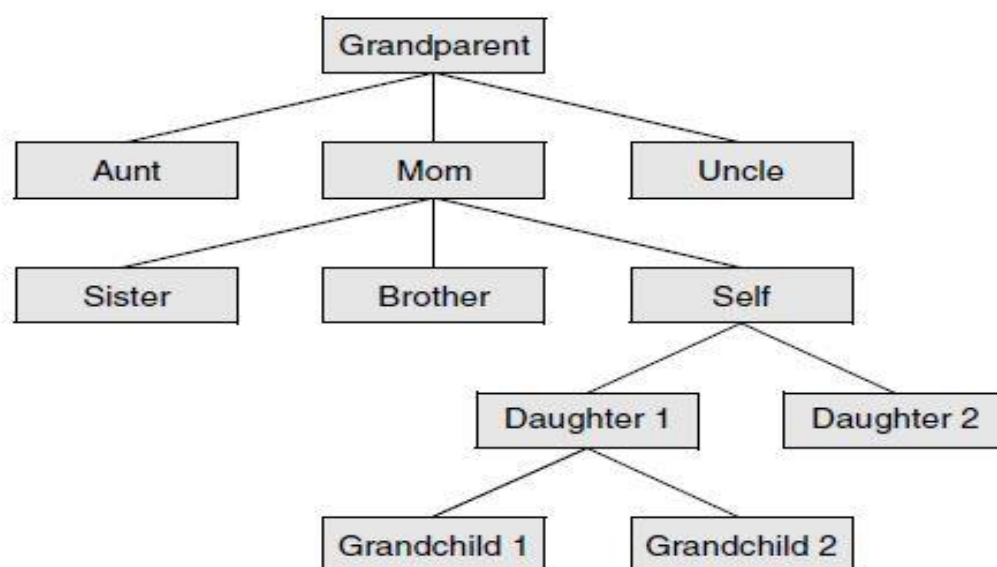
write: write: you have write permission turned off.

### **Email**

Email enables you to communicate with users on the local system and, if the installation is part of a network, with other users on the network. If you are connected to the Internet, you can communicate electronically with users around the world. Email utilities differ from `write` in that email utilities can send a message when the recipient is not logged in. In this case the email is stored until the recipient reads it. These utilities can also send the same message to more than one user at a time. Many email programs are available for Linux, including the original character-based `mailx` program, Mozilla/Thunderbird, `pine`, `mail` through `emacs`, `KMail`, and `evolution`. Another popular graphical email program is `sylpheed`. Two programs are available that can make any email program easier to use and more secure. The `procmail` program creates and maintains email servers and mailing lists; preprocesses email by sorting it into appropriate files and directories. The GNU Privacy Guard encrypts and decrypts email and makes it almost impossible for an unauthorized person to read.

## **6.3 The Filesystem**

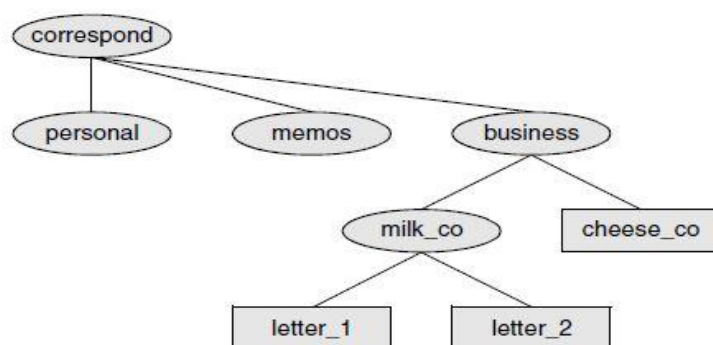
A filesystem is a set of data structures that usually resides on part of a disk and that holds directories of files. Filesystems store user and system data that are the basis of users’ work on the system and the system’s existence. A hierarchical structure frequently takes the shape of a pyramid. One example: a family’s lineage. A couple has a child, who may in turn have several children, each of whom may have more children. This hierarchical structure is called a family tree.



Like the family tree it resembles, the Linux filesystem is called a tree. It consists of a set of connected files. This structure allows you to organize files so you can easily find any one. On a standard Linux system, each user starts with one directory, to which the user can add subdirectories to any desired level. By creating multiple levels of subdirectories, a user can expand the structure as needed. Typically each subdirectory is dedicated to a single subject, such as a person, project, or event. The subject dictates whether a subdirectory should be subdivided further. For example, a secretary's subdirectory named correspond.



Example: A secretary's directories



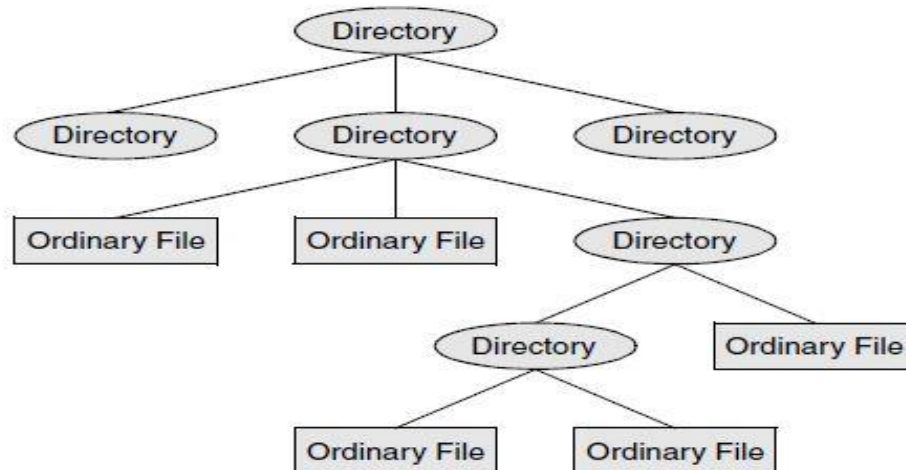
This directory contains three subdirectories: business, memos, and personal. The business directory contains files that store each letter the secretary types.

### Strength of Linux File System

One major strength of the Linux filesystem is its ability to adapt to users' needs. You can take advantage of this strength by strategically organizing your files so they are most convenient and useful for you.

## Directory Files and Ordinary Files

Like a family tree, the tree representing the filesystem is usually pictured upside down, with its root at the top. The tree “grows” downward from the root, with paths connecting the root to each of the other files. At the end of each path is either an ordinary file or a directory file. Ordinary files, or simply files, appear at the ends of paths that cannot support other paths. Directory files, also referred to as directories or folders, are the points that other paths can branch off from. When you refer to the tree, up is toward the root and down is away from the root. Directories directly connected by a path are called parents (closer to the root) and children (farther from the root). A pathname is a series of names that trace a path along branches from one file to another.



### Filename

Every file has a filename. The maximum length of a filename varies with the type of filesystem; Linux supports several types of filesystems. Although most of today’s filesystems allow files with names up to 255 characters long, some filesystems restrict filenames to fewer characters. Choose characters from the following list:

- Uppercase letters (A–Z)
- Lowercase letters (a–z)
- Numbers (0–9)
- Underscore ( \_ )
- Period ( . )
- Comma ( , )

Like the children of one parent, no two files in the same directory can have the same name. Files in different directories, like the children of different parents, can have the same name. The filenames you choose should mean something. Too often a directory is filled with important files with such unhelpful names as hold1, wombat, and junk, not to mention foo and foobar. Such names are poor choices because they do not help you recall what you stored in a file. The following filenames conform to the suggested syntax and convey information about the contents of the file:

- Correspond
- January
- Davis
- Reports
- 2001
- acct\_payable

**Unit 06: File Systems**

If you keep the filenames short, they are easy to type. The disadvantage of short filenames is that they are typically less descriptive than long filenames. Long filenames enable you to assign descriptive names to files. To help you select among files without typing entire filenames, shells support filename completion. You can use uppercase and/or lowercase letters within filenames, but be careful: Many filesystems are case sensitive. For example, the popular ext family of filesystems and the UFS filesystem are case sensitive, so files named JANUARY, January, and January refer to three distinct files. The FAT family of filesystems (used mostly for removable media) is not case sensitive, so those three filenames represent the same file. The HFS+ filesystem, which is the default OS X filesystem, is case preserving but not case sensitive. Although you can use SPACES within filenames, it is a poor idea. Because a SPACE is a special character, you must quote it on a command line. Use periods or underscores instead of SPACES: joe.05.04.26, new\_stuff.

```
$ lpr my\ file
```

```
$ lpr "my file"
```

**Filename Extensions**

A filename extension is the part of the filename following an embedded period. The filename extensions help describe the contents of the file.

| Filename with Extension | Meaning of Extension                                                     |
|-------------------------|--------------------------------------------------------------------------|
| compute.c               | A C programming language source file                                     |
| compute.o               | The object code file of compute.c                                        |
| compute                 | The executable file of compute.c                                         |
| memo-0410.txt           | A text file                                                              |
| memo.pdf                | A PDF file, view with xpdf or kpdf                                       |
| memo.ps                 | A PostScript file, view with gs or kpdf                                  |
| memo.z                  | A file compressed with compress; use uncompress or gunzip to decompress  |
| Memo.tgz or memo.tar.gz | A tar archive of files compressed with gzip                              |
| Memo.gz                 | A file compressed with gzip, view with zcat or decompress with gunzip    |
| Memo.bz2                | A file compressed with bzip2; view with bzip2 or decompress with bunzip2 |
| Memo.html               | A file meant to be viewed using a Web browser, such as firefox           |

|                                        |                                                          |
|----------------------------------------|----------------------------------------------------------|
| .gif, .jpg, .jpeg, .bmp, .tif or .tiff | A file containing graphical information, such as picture |
|----------------------------------------|----------------------------------------------------------|

### Hidden Filenames

A filename that begins with a period is called a hidden filename because `ls` does not normally display it. The command `ls -a` displays all filenames, even hidden ones. Names of startup files usually begin with a period so that they are hidden and do not clutter a directory listing. The `.plan` file is also hidden. Two special hidden entries—a single and double period (`.` and `..`)—appear in every directory.

### The Working Directory

While you are logged in on a Linux system, you are always associated with a directory. The directory you are associated with is called the working directory or current directory. Sometimes this association is referred to in a physical sense: “You are in (or working in) the `zach` directory.” The `pwd` (print working directory) shell builtin displays the pathname of the working directory.

### Your Home Directory

When you first log in, the working directory is your home directory. To display the pathname of your home directory, use `pwd` just after you log in. Linux home directories are typically located in `/home`. When used without any arguments, the `ls` utility displays a list of the files in the working directory. All the files you have created up to this point were created in your home directory.

### Startup Files

Startup files, which appear in your home directory, give the shell and other programs information about you and your preferences. Because the startup files have hidden filenames, you must use the `ls -a` command to see whether one is in your home directory.

## 6.4 Pathnames

Every file has a pathname, which is a trail from a directory through part of the directory hierarchy to an ordinary file or a directory. Within a pathname, a slash (`/`) to the right of a filename indicates that the file is a directory file. The file following the slash can be an ordinary file or a directory file.

- Absolute Pathnames
- Relative Pathnames

### Absolute Pathnames

The root directory of the filesystem hierarchy does not have a name. It is referred to as the root directory. It is represented by a `/` (slash) standing alone or at the left end of a pathname. An absolute pathname starts with a slash (`/`), which represents the root directory. The slash is followed by the name of a file located in the root directory. An absolute pathname continues, tracing a path through all intermediate directories, to the file identified by the pathname. String all the filenames in the path together, following each directory with a slash (`/`). This string of filenames is called an absolute pathname because it locates a file absolutely by tracing a path from the root directory to the file. Using an absolute pathname, you can list or otherwise work with any file on the local system, regardless of the working directory at the time you give the command. For example, Sam can give the following command while working in his home directory to list the files in the `/usr/bin` directory:

```
$ pwd
/home/sam
```

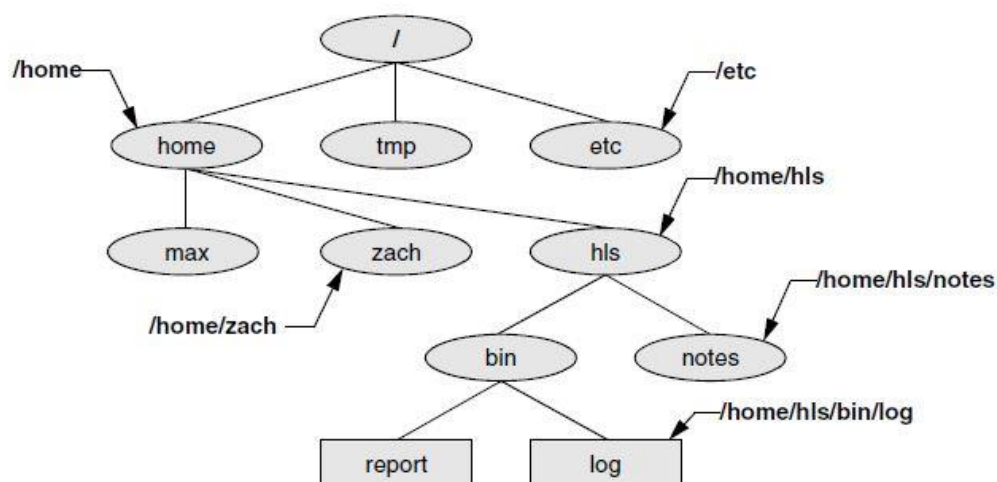


```
$ ls /usr/bin
```

```
7z kwin
```

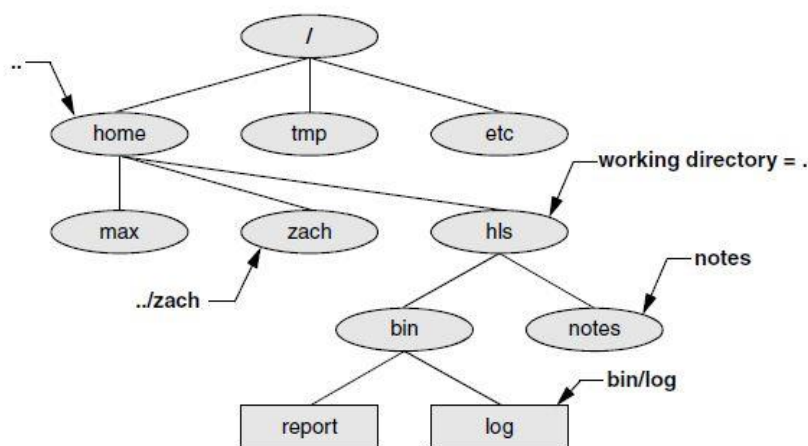
```
7za kwin_killer_helper
```

```
822-date kwin_rules_dialog
```



## Relative Pathnames

A relative pathname traces a path from the working directory to a file. The pathname is relative to the working directory. Any pathname that does not begin with the root directory (represented by /) or a tilde (~) is a relative pathname. Like absolute pathnames, relative pathnames can trace a path through many directories. The simplest relative pathname is a simple filename, which identifies a file in the working directory.



## Significance of the Working Directory

To access any file in the working directory, you need only a simple filename. To access a file in another directory, you must use a pathname. Typing a long pathname is tedious and increases the chance of making a mistake. This possibility is less likely under a GUI, where you click filenames or icons. You can choose a working directory for any task to reduce the need for long pathnames.

## 6.5 Working with Directories

- how to create directories (mkdir),

### *Linux and Shell Scripting*

---

- switch between directories (cd),
- remove directories (rmdir)
- mv, cp: Move or Copy Files
- mv: Moves a Directory

### **mkdir: Creates a Directory**

The mkdir utility creates a directory. The argument to mkdir becomes the pathname of the new directory.

```
$ pwd
/home/max
$ ls
demo names temp
$ mkdir literature
$ ls
demo literature names temp
$ ls -F
demo literature/ names temp
$ ls literature
$
```

He uses a relative pathname (a simple filename) because he wants the literature directory to be a child of the working directory. Max could have used an absolute pathname to create the same directory: mkdir /home/max/literature or mkdir ~max/literature. There are two ways to create the promo directory as a child of the newly created literature directory.

**The first way checks that /home/max is the working directory and uses a relative pathname:**

```
$ pwd
/home/max
$ mkdir literature/promo
```

**The second way uses an absolute pathname:**

```
$ mkdir /home/max/literature/promo
```

### **cd: Changes to Another Working Directory**

The cd (change directory) utility makes another directory the working directory but does not change the contents of the working directory.

```
$ cd /home/max/literature
$ pwd
/home/max/literature
$ cd
$ pwd
/home/max
$ cd literature
$ pwd
/home/max/literature
```

### The working directory versus your home directory

The working directory is not the same as your home directory. Your home directory remains the same for the duration of your session and usually from session to session. Immediately after you log in, you are always working in the same directory: your home directory. Unlike your home directory, the working directory can change as often as you like. You have no set working directory, which explains why some people refer to it as the current directory. When you log in and until you change directories using `cd`, your home directory is the working directory.

### The . and .. Directory Entries

The `mkdir` utility automatically puts two entries in each directory it creates: a single period (.) and a double period (..). The . is synonymous with the pathname of the working directory and can be used in its place; the .. is synonymous with the pathname of the parent of the working directory. These entries are hidden because their filenames begin with a period.

### rmdir: Deletes a Directory

The `rmdir` (remove directory) utility deletes a directory. You cannot delete the working directory or a directory that contains files other than the . and .. entries. If you need to delete a directory that has files in it, first use `rm` to delete the files and then delete the directory. You do not have to (nor can you) delete the . and .. entries; `rmdir` removes them automatically. The following command deletes the `promo` directory:

```
$ rmdir /home/max/literature/promo
```

The `rm` utility has a `-r` option (`rm -r filename`) that recursively deletes files, including directories, within a directory and also deletes the directory itself.

Although `rm -r` is a handy command, you must use it carefully. Do not use it with an ambiguous file reference such as `*`. It is shockingly easy to wipe out your entire home directory with a single short command.

### mv, cp: Move or Copy Files

You can use this utility to move files from one directory to another (change the pathname of a file) as well as to change a simple filename. When used to move one or more files to a new directory, the `mv` command has this syntax:

```
mv existing-file-list directory.
```

If the working directory is `/home/max`, Max can use the following command to move the files `names` and `temp` from the working directory to the `literature` directory:

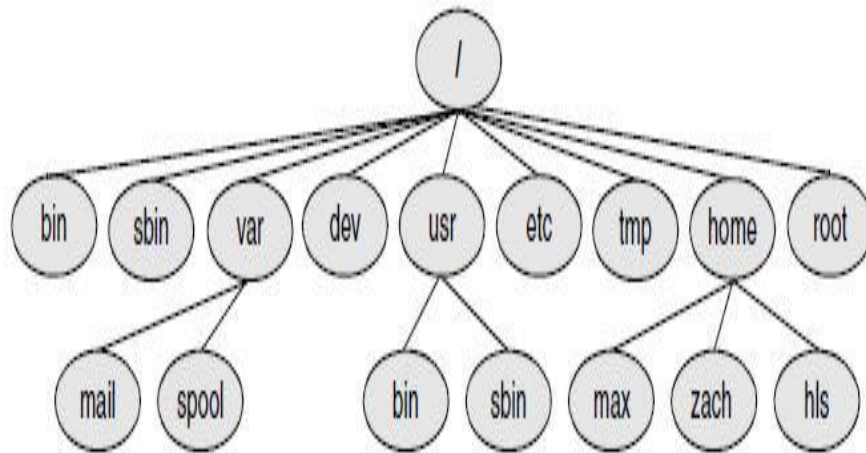
```
$ mv names temp literature.
```

This command changes the absolute pathnames of the `names` and `temp` files from `/home/max/names` and `/home/max/temp` to `/home/max/literature/names` and `/home/max/literature/temp`, respectively. Like most Linux commands, `mv` accepts either absolute or relative pathnames. The `cp` utility works in the same way as `mv` does, except that it makes copies of the existing-file-list in the specified directory.

**mv: Moves a Directory**

Just as it moves ordinary files from one directory to another, so mv can move directories. The syntax is similar except that you specify one or more directories, not ordinary files, to move: `mv existing-directory-list new-directory`

If new-directory does not exist, the existing-directory-list must contain just one directory name, which mv changes to new-directory (mv renames the directory). Although you can rename directories using mv, you cannot copy their contents with cp unless you use the `-r` (recursive) option.

**A typical FHS-based Linux filesystem structure****Important Standard Directories and Files**

|          |                                                                                                                                                                                                                           |
|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| /        | Root: The root directory, present in all Linux filesystem structures, is the ancestor of all files in the filesystem.                                                                                                     |
| /bin     | Essential command binaries Holds the files needed to bring the system up and run it when it first comes up in single-user or recovery mode.                                                                               |
| /boot    | Static files of the boot loader Contains all files needed to boot the system.                                                                                                                                             |
| /dev     | Device files Contains all files that represent peripheral devices, such as disk drives, terminals, and printers                                                                                                           |
| /etc     | Machine-local system configuration files Holds administrative, configuration, and other system files. One of the most important is /etc/passwd, which contains a list of all users who have permission to use the system. |
| /etc/opt | Configuration files for add-on software packages kept in /opt                                                                                                                                                             |
| /etc/X11 | Machine-local configuration files for the X Window System                                                                                                                                                                 |

*Unit 06: File Systems*

|                 |                                                                                                                                                                                                                                                        |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| /home           | User home directories                                                                                                                                                                                                                                  |
| /lib            | Shared libraries                                                                                                                                                                                                                                       |
| /lib/modules    | Loadable kernel modules                                                                                                                                                                                                                                |
| /mnt            | Mount point for temporarily mounting filesystems                                                                                                                                                                                                       |
| /opt            | Add-on (optional) software packages                                                                                                                                                                                                                    |
| /proc           | Kernel and process information virtual filesystem                                                                                                                                                                                                      |
| /root           | Home directory for the root account                                                                                                                                                                                                                    |
| /sbin           | Essential system binaries Utilities used for system administration are stored in /sbin and /usr/sbin. The /sbin directory includes utilities needed during the booting process, and /usr/sbin holds utilities used after the system is up and running. |
| /sys            | Device pseudofilesystem                                                                                                                                                                                                                                |
| /tmp            | Temporary Files                                                                                                                                                                                                                                        |
| /usr/games      | Games and educational programs                                                                                                                                                                                                                         |
| /usr/include    | Header files used by C programs                                                                                                                                                                                                                        |
| /usr/lib        | Libraries                                                                                                                                                                                                                                              |
| /usr/local      | Local hierarchy Holds locally important files and directories that are added to the system. Subdirectories can include bin, games, include, lib, sbin, share, and src.                                                                                 |
| /usr/sbin       | Nonvital system administration binaries See /sbin.                                                                                                                                                                                                     |
| /usr/share      | Architecture-independent data Subdirectories can include dict, doc, games, info, locale, man, misc, terminfo, and zoneinfo.                                                                                                                            |
| /usr/share/doc  | Documentation                                                                                                                                                                                                                                          |
| /usr/share/info | GNU info system's primary directory                                                                                                                                                                                                                    |

|                             |                                                                                                                                                                                                                                             |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>/usr/share/man</code> | Online manuals                                                                                                                                                                                                                              |
| <code>/usr/src</code>       | Source code                                                                                                                                                                                                                                 |
| <code>/var</code>           | <b>Variable data</b> Files with contents that vary as the system runs are kept in subdirectories under <code>/var</code> . The most common examples are temporary files, system log files, spooled files, and user mailbox files.           |
| <code>/var/log</code>       | <b>Log files</b> Contains <code>lastlog</code> (a record of the last login by each user), <code>messages</code> (system messages from <code>syslogd</code> ), and <code>wtm</code> (a record of all logins/logouts), among other log files. |
| <code>/var/spool</code>     | <b>Spooled application data</b> Contains <code>anacron</code> , <code>at</code> , <code>cron</code> , <code>lpd</code> , <code>mail</code> , <code>mqueue</code> , <code>samba</code> , and other directories.                              |

## 6.6 Linux Access Permissions

Linux supports two methods of controlling who can access a file and how they can access it:

- 1) Traditional Linux access permissions
- 2) Access Control Lists (ACLs).

### Three types of users can access a file:

- 1) The owner of the file (owner),
- 2) A member of a group that the file is associated with (group),
- 3) Everyone else (other).

### Ways to Access an Ordinary File:

- A user can attempt to access an ordinary file in three ways:
  - 1) Read from
  - 2) Write to
  - 3) Execute it.

### Access Permissions

- `ls -l`: Displays Permissions
- `chmod`: Changes Access Permissions
- `Setuid` and `Setgid` Permissions
- Directory Access Permissions

### `ls -l`: Displays Permissions:

When you call `ls` with the `-l` option and the name of one or more ordinary files, `ls` displays a line of information about the file.

```

divya@localhost:~
File Edit View Terminal Go Help
[divya@localhost divya]$ ls
Syllabus
[divya@localhost divya]$ ls -l Syllabus
-rw-rw-r--  1 divya  divya    455 Mar 22 09:36 Syllabus
[divya@localhost divya]$

```

From left to right, the lines that an `ls -l` command displays contain: The type of file (first character), the file's access permissions (the next nine characters), the ACL flag (present if the file has an ACL), the number of links to the file, the name of the owner of the file (usually the person who created the file), the name of the group the file is associated with, the size of the file in characters (bytes), the date and time the file was created or last modified, the name of the file and the type of file is a hyphen (-) because it is an ordinary file.

| Character | Meaning           |
|-----------|-------------------|
| -         | Ordinary          |
| b         | Block Device      |
| c         | Character Device  |
| d         | Directory         |
| p         | FIFO (names Pipe) |
| l         | Symbolic link     |

The next three characters specify the access permissions for the owner of the file.

- R indicates read permission,
- w indicates write permission,

## Linux and Shell Scripting

- x indicates execute permission.

A - in one of the positions indicates that the owner does not have the permission associated with that position. In a similar manner the next three characters represent permissions for the group. The final three characters represent permissions for other (everyone else). Although execute permission can be allowed for any file, it does not make sense to assign execute permission to a file that contains a document, such as a letter.

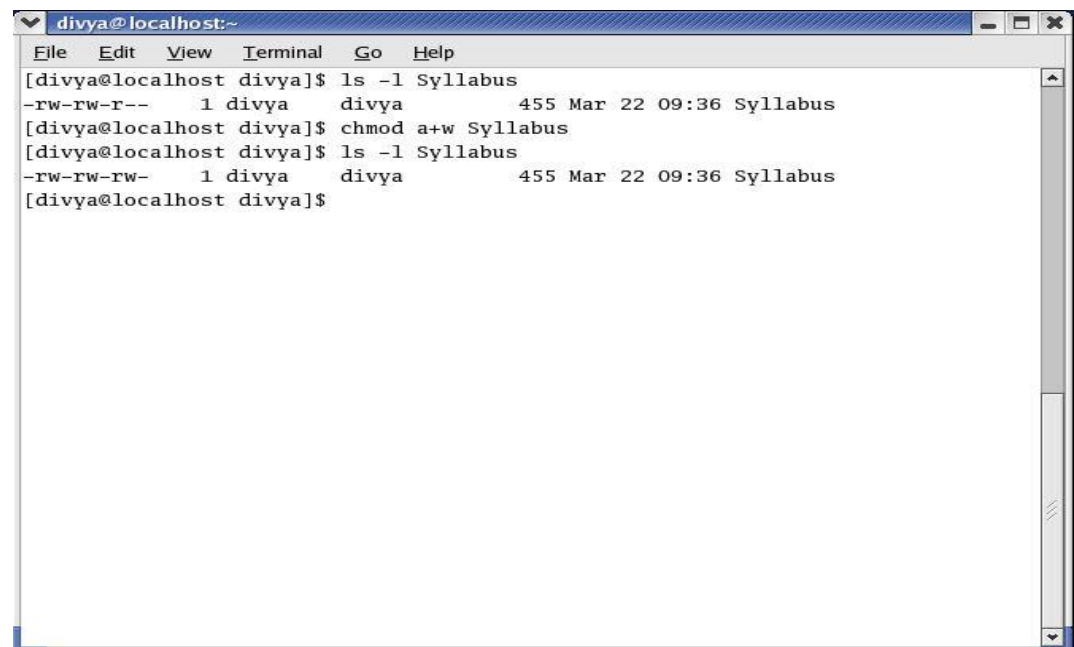
### chmod: Changes Access Permissions

The Linux file access permission scheme lets you give other users access to the files you want to share yet keep your private files confidential.

- You can allow other users to read from and write to a file.
- You can also allow others only to read from a file.
- Or you can allow others only to write to a file.

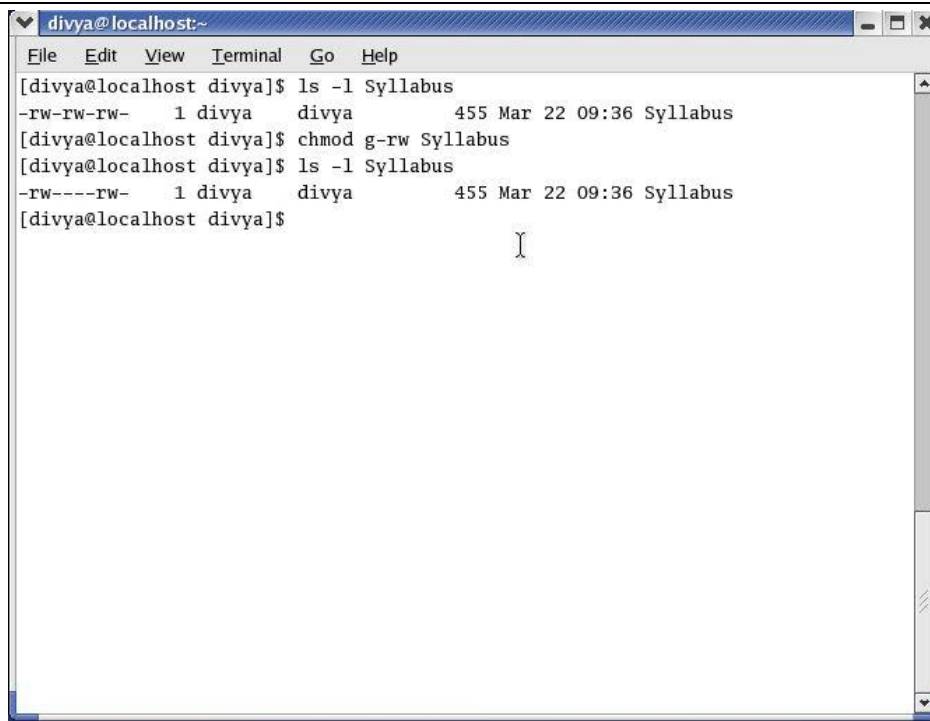
A user with root privileges can access any file on the system. Anyone who can gain root privileges has full access to all files, regardless of the file's owner or access permissions. The owner of a file controls which users have permission to access the file and how those users can access it. When you own a file, you can use the chmod (change mode) utility to change access permissions for that file. You can specify symbolic (relative) or numeric (absolute) arguments to chmod.

### Symbolic Arguments to chmod



```
divya@localhost:~  
File Edit View Terminal Go Help  
[divya@localhost divya]$ ls -l Syllabus  
-rw-rw-r-- 1 divya divya 455 Mar 22 09:36 Syllabus  
[divya@localhost divya]$ chmod a+w Syllabus  
[divya@localhost divya]$ ls -l Syllabus  
-rw-rw-rw- 1 divya divya 455 Mar 22 09:36 Syllabus  
[divya@localhost divya]$
```



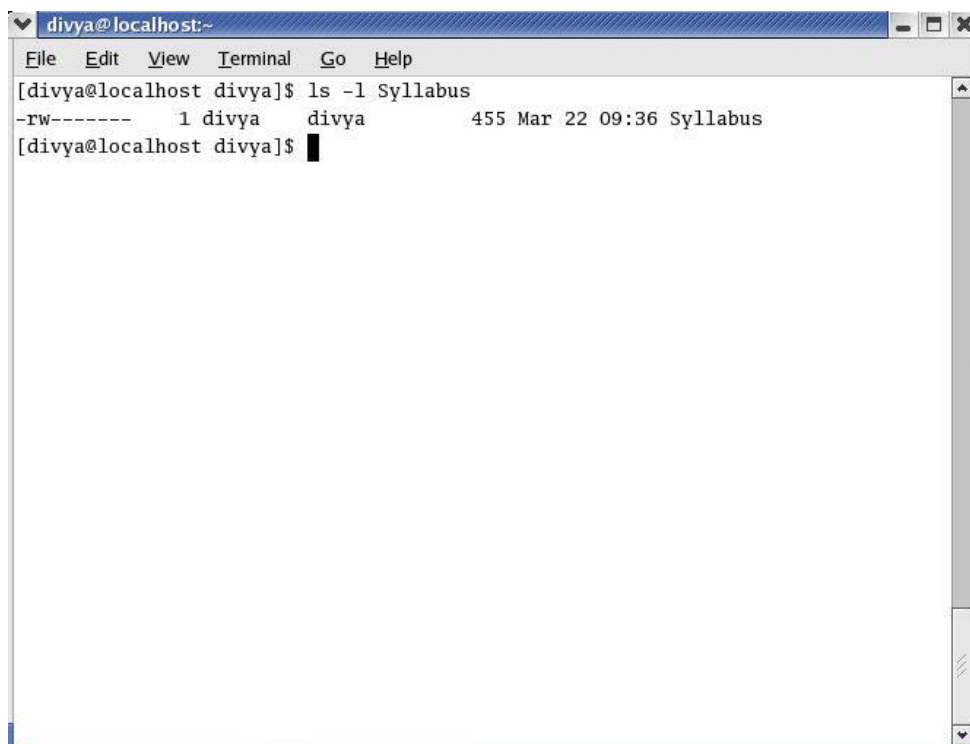


```
divya@localhost:~  
File Edit View Terminal Go Help  
[divya@localhost divya]$ ls -l Syllabus  
-rw-rw-rw- 1 divya divya 455 Mar 22 09:36 Syllabus  
[divya@localhost divya]$ chmod g-rw Syllabus  
[divya@localhost divya]$ ls -l Syllabus  
-rw----rw- 1 divya divya 455 Mar 22 09:36 Syllabus  
[divya@localhost divya]$
```

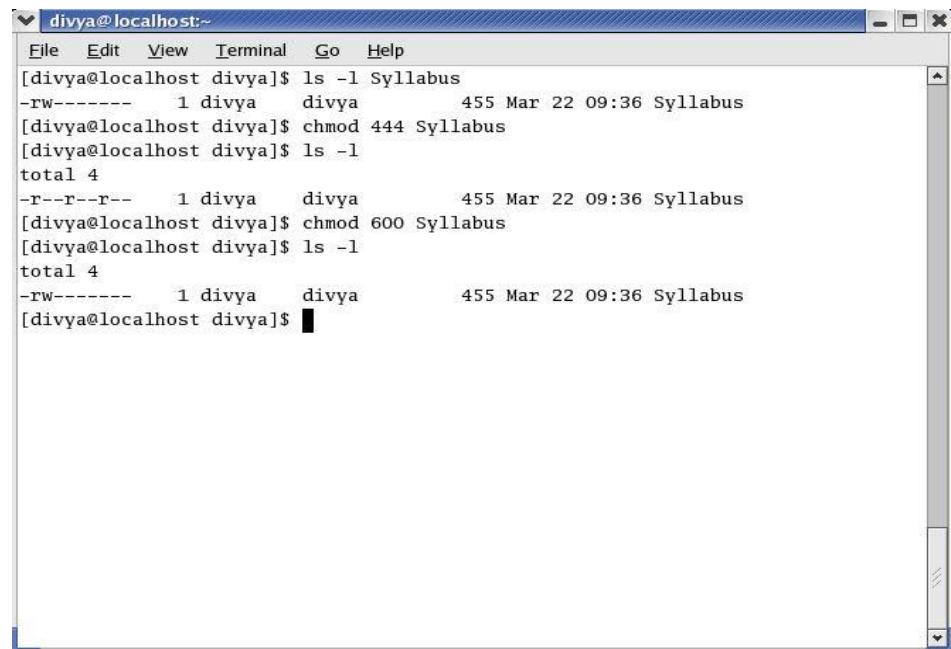
When using chmod, many people assume that the o stands for owner; it does not. The o stands for other, whereas u stands for owner (user).UGO (user-group-other)

### Numeric Arguments to chmod

You can also use numeric arguments to specify permissions with chmod. In place of the letters and symbols specifying permissions used, here numeric arguments comprise three octal digits. The first digit specifies permissions for the owner, the second for the group, and the third for other users. A 1 gives the specified user(s) execute permission, 2 gives write permission, and 4 gives read permission. Construct the digit representing the permissions for the owner, group, or others by ORing (adding) the appropriate values.



```
divya@localhost:~  
File Edit View Terminal Go Help  
[divya@localhost divya]$ ls -l Syllabus  
-rw----- 1 divya divya 455 Mar 22 09:36 Syllabus  
[divya@localhost divya]$
```

A terminal window titled 'divya@localhost:~' with a menu bar (File, Edit, View, Terminal, Go, Help). The user runs 'ls -l Syllabus', showing '-rw----- 1 divya divya 455 Mar 22 09:36 Syllabus'. Then 'chmod 444 Syllabus' is run. Then 'ls -l' is run, showing 'total 4' and '-r--r--r-- 1 divya divya 455 Mar 22 09:36 Syllabus'. Then 'chmod 600 Syllabus' is run. Finally 'ls -l' is run, showing 'total 4' and '-rw----- 1 divya divya 455 Mar 22 09:36 Syllabus'.

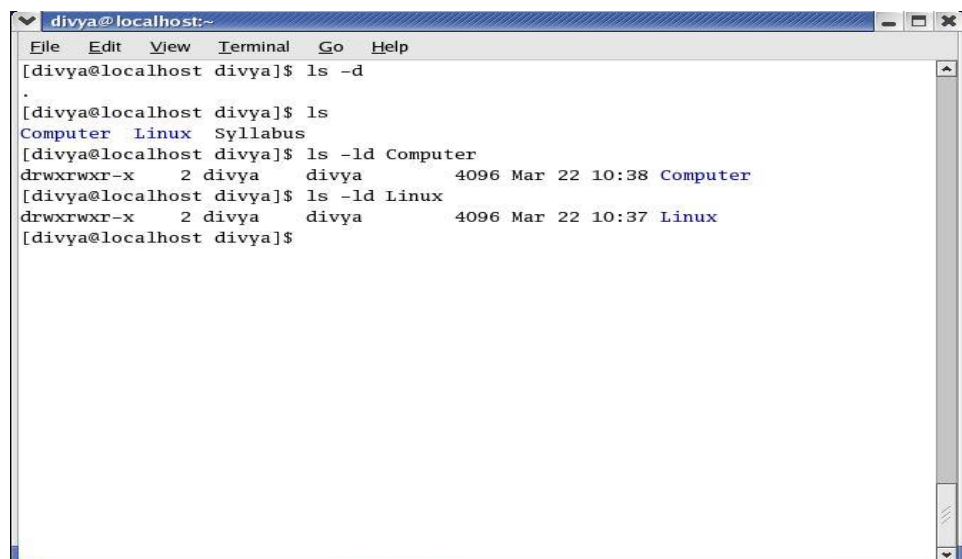
```
divya@localhost:~  
File Edit View Terminal Go Help  
[divya@localhost divya]$ ls -l Syllabus  
-rw----- 1 divya divya 455 Mar 22 09:36 Syllabus  
[divya@localhost divya]$ chmod 444 Syllabus  
[divya@localhost divya]$ ls -l  
total 4  
-r--r--r-- 1 divya divya 455 Mar 22 09:36 Syllabus  
[divya@localhost divya]$ chmod 600 Syllabus  
[divya@localhost divya]$ ls -l  
total 4  
-rw----- 1 divya divya 455 Mar 22 09:36 Syllabus  
[divya@localhost divya]$
```

### Setuid and Setgid Permissions

When you execute a file that has setuid (set user ID) permission, the process executing the file takes on the privileges of the file's owner. For example, if you run a setuid program that removes all files in a directory, you can remove files in any of the file owner's directories, even if you do not normally have permission to do so. In a similar manner, setgid (set group ID) permission gives the process executing the file the privileges of the group the file is associated with.

### Directory Access Permissions

Access permissions have slightly different meanings when they are used with directories. Although the three types of users can read from or write to a directory, the directory cannot be executed. Execute permission is redefined for a directory: It means that you can cd into the directory and/or examine files that you have permission to read from in the directory. It has nothing to do with executing a file. When you have only execute permission for a directory, you can use ls to list a file in the directory if you know its name. You cannot use ls without an argument to list the entire contents of the directory.

A terminal window titled 'divya@localhost:~' with a menu bar (File, Edit, View, Terminal, Go, Help). The user runs 'ls -d', showing '.'. Then 'ls' is run, showing 'Computer Linux Syllabus'. Then 'ls -ld Computer' is run, showing 'drwxrwxr-x 2 divya divya 4096 Mar 22 10:38 Computer'. Then 'ls -ld Linux' is run, showing 'drwxrwxr-x 2 divya divya 4096 Mar 22 10:37 Linux'.

```
divya@localhost:~  
File Edit View Terminal Go Help  
[divya@localhost divya]$ ls -d  
.  
[divya@localhost divya]$ ls  
Computer Linux Syllabus  
[divya@localhost divya]$ ls -ld Computer  
drwxrwxr-x 2 divya divya 4096 Mar 22 10:38 Computer  
[divya@localhost divya]$ ls -ld Linux  
drwxrwxr-x 2 divya divya 4096 Mar 22 10:37 Linux  
[divya@localhost divya]$
```

The `d` at the left end of the line that `ls` displays indicates that `Linux` is a directory. The person has read, write, and execute permissions; members of the group also have read, write and execute permissions; and other users have only read and execute permissions.

## 6.7 Access Control Lists

The ACLs provide finer-grained control over which users can access specific directories and files than do traditional Linux permissions. Using ACLs you can specify the ways in which each of several users can access a directory or file.

### Reduced Performance

Because ACLs can reduce performance, do not enable them on filesystems that hold system files, where the traditional Linux permissions are sufficient. Also be careful when moving, copying, or archiving files: Not all utilities preserve ACLs. In addition, you cannot copy ACLs to filesystems that do not support ACLs.

### Rules

An ACL comprises a set of rules. **Rule:** It specifies how a specific user or group can access the file that the ACL is associated with. There are two kinds of rules: **access rules** and **default rules**.

- **Access rule:** It specifies access information for a single file or directory.
- **Default ACL:** It pertains to a directory only; it specifies default access information (an ACL) for any file in the directory that is not given an explicit ACL.

### Most utilities do not preserve ACLs

- **cp utility:** When used with the `-p` (preserve) or `-a` (archive) option, `cp` preserves ACLs when it copies files.
- **mv utility:** It also preserves ACLs.
- Other utilities, such as `tar`, `cpio`, and `dump`, do not support ACLs.

### Enabling ACLs

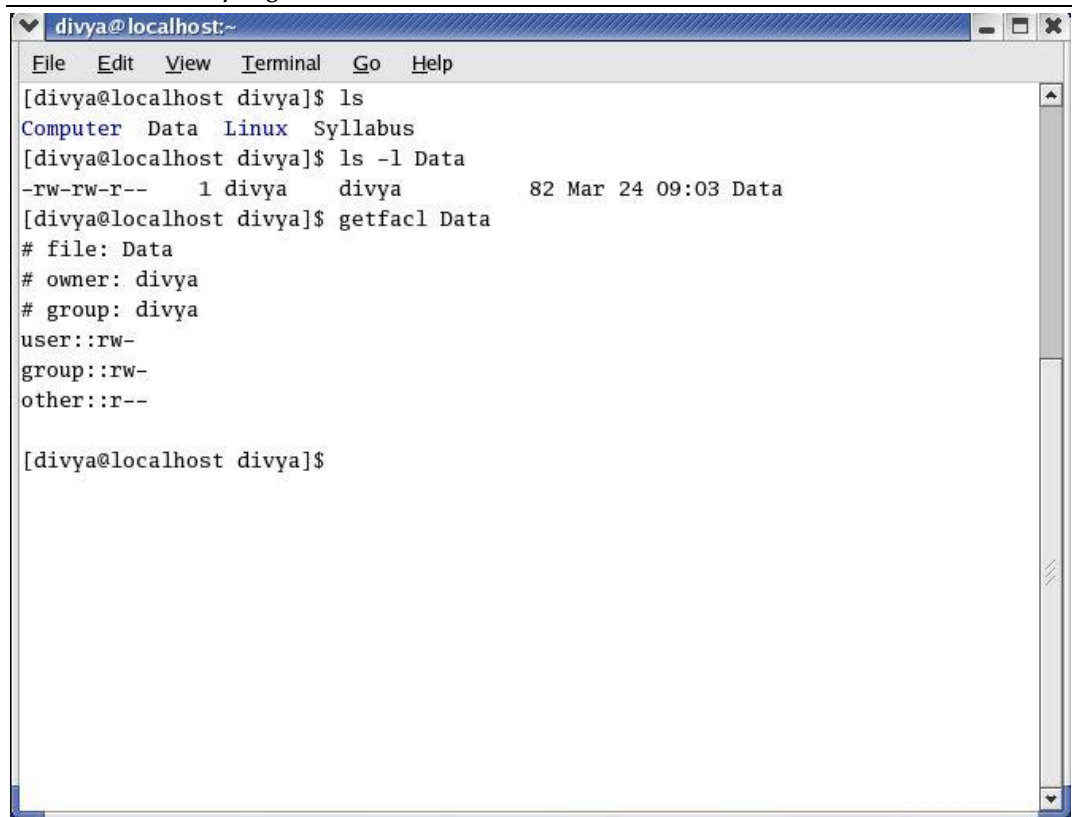
Before you can use ACLs you must install the `acl` software package. Linux officially supports ACLs on `ext2`, `ext3`, and `ext4` filesystems only, although informal support for ACLs is available on other filesystems. To use ACLs on an `ext` filesystem, you must mount the device with the `acl` option. For example, if you want to mount the device represented by `/home` so you can use ACLs on files in `/home`, you can add `acl` to its options list in `/etc/fstab`:

```
$ grep home /etc/fstab
```

```
LABEL=/home /home ext3 defaults,acl 1 2
```

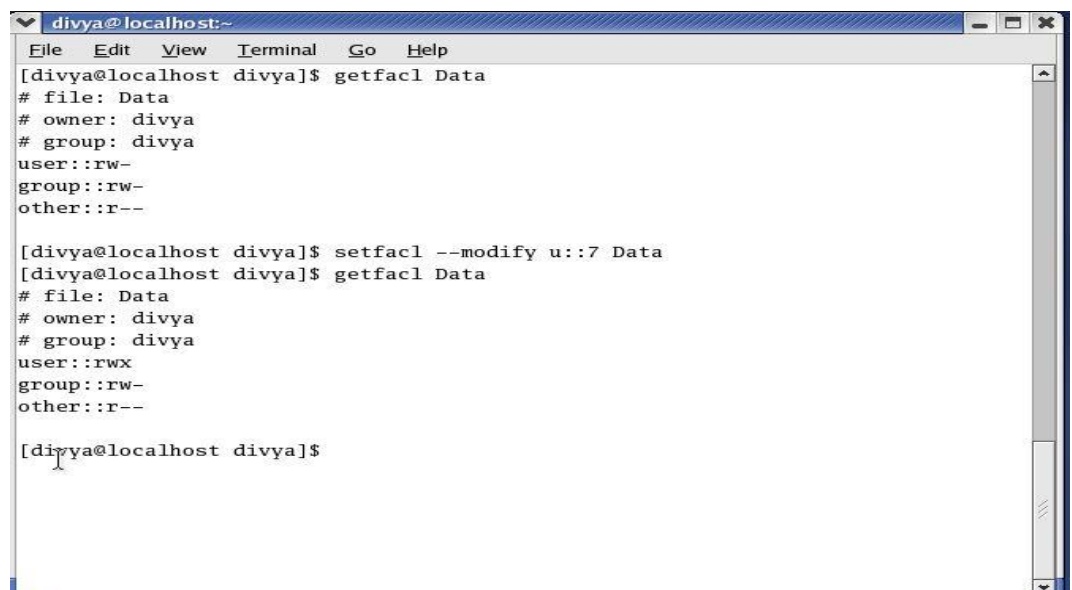
After changing `fstab`, you need to remount `/home` before you can use ACLs.

## Linux and Shell Scripting

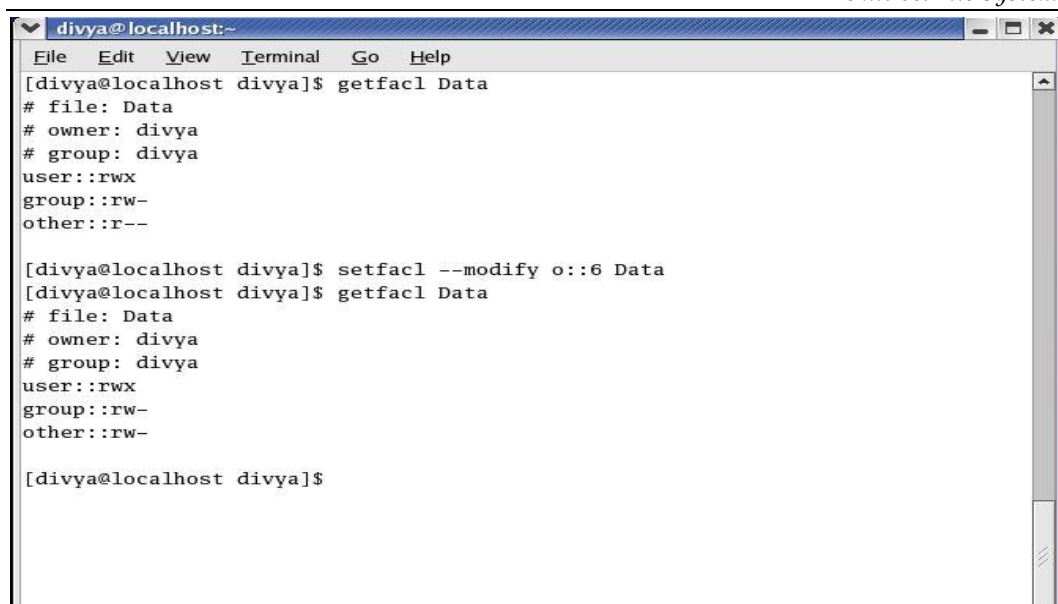


```
divya@localhost:~  
File Edit View Terminal Go Help  
[divya@localhost divya]$ ls  
Computer Data Linux Syllabus  
[divya@localhost divya]$ ls -l Data  
-rw-rw-r-- 1 divya divya 82 Mar 24 09:03 Data  
[divya@localhost divya]$ getfacl Data  
# file: Data  
# owner: divya  
# group: divya  
user::rw-  
group::rw-  
other::r--  
  
[divya@localhost divya]$
```

The first three lines of the `getfacl` output comprise the header; they specify the name of the file, the owner of the file, and the group the file is associated with. In the line that starts with `user`, the two colons (:) with no name between them indicate that the line specifies the permissions for the owner of the file. Similarly, the two colons in the `group` line indicate that the line specifies permissions for the group the file is associated with. The two colons following `other` are there for consistency: No name can be associated with `other`.



```
divya@localhost:~  
File Edit View Terminal Go Help  
[divya@localhost divya]$ getfacl Data  
# file: Data  
# owner: divya  
# group: divya  
user::rw-  
group::rw-  
other::r--  
  
[divya@localhost divya]$ setfacl --modify u::7 Data  
[divya@localhost divya]$ getfacl Data  
# file: Data  
# owner: divya  
# group: divya  
user::rwx  
group::rw-  
other::r--  
  
[divya@localhost divya]$
```



```
divya@localhost:~
File Edit View Terminal Go Help
[divya@localhost divya]$ getfacl Data
# file: Data
# owner: divya
# group: divya
user::rwx
group::rw-
other::r--

[divya@localhost divya]$ setfacl --modify o::6 Data
[divya@localhost divya]$ getfacl Data
# file: Data
# owner: divya
# group: divya
user::rwx
group::rw-
other::rw-

[divya@localhost divya]$
```

### Setting up ACL :

1) To add permission for user

**setfacl -m "u:user:permissions" /path/to/file**

2) To add permissions for a group

**setfacl -m "g:group:permissions" /path/to/file**

3) To allow all files or directories to inherit ACL entries from the directory it is within

**setfacl -dm "entry" /path/to/dir**

4) To remove a specific entry

**setfacl -x "entry" /path/to/file**

5) To remove all entries

**setfacl -b path/to/file**

### Modifying ACL using setfacl:

To add permissions for a user (user is either the user name or ID): # **setfacl -m "u:user:permissions"**. To add permissions for a group (group is either the group name or ID): # **setfacl -m "g:group:permissions"**. To allow all files or directories to inherit ACL entries from the directory it is within: # **setfacl -dm "entry"**.

### View ACL :

To show permissions : # **getfacl filename**

### Remove ACL :

If you want to remove the set ACL permissions, use setfacl command with **-b** option.

### Setting Default Rules for a Directory

The setfacl command uses the **-d** (default) option to add two default rules to the ACL for dir. These rules apply to all files in the dir directory that do not have explicit ACLs.

```

divya@localhost:~
File Edit View Terminal Go Help
[divya@localhost divya]$ ls
Computer Data Linux Syllabus
[divya@localhost divya]$ ls -ld Computer
drwxrwxr-x  2 divya  divya    4096 Mar 22 10:38 Computer
[divya@localhost divya]$ getfacl Computer/
# file: Computer
# owner: divya
# group: divya
user::rwx
group::rwx
other::r-x

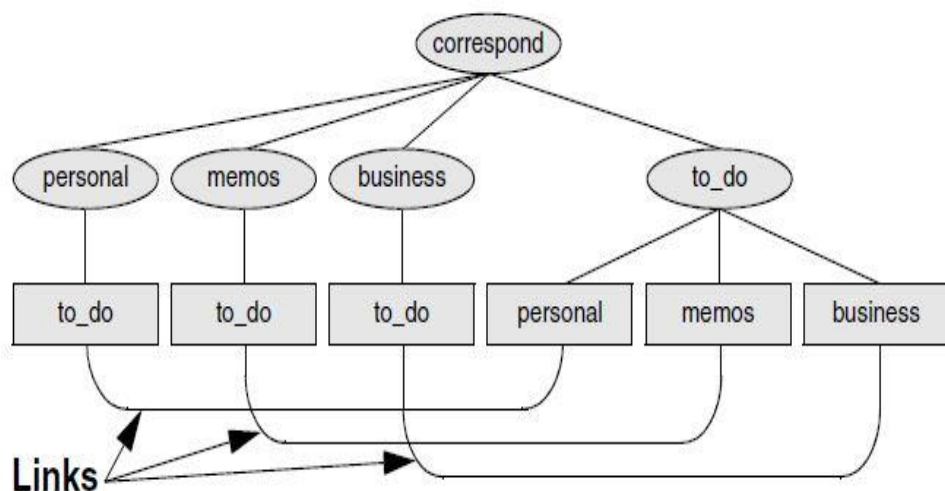
[divya@localhost divya]$

```

## 6.8 Links

A link is a pointer to a file. Every time you create a file by using vim, touch, cp, or any other means, you are putting a pointer in a directory. This pointer associates a filename with a place on the disk. When you specify a filename in a command, you are indirectly pointing to the place on the disk that holds the information you want.

### Using links to cross-classify files



## Using links

Sharing files can be useful when two or more people are working on the same project and need to share some information. You can make it easy for other users to access one of your files by creating additional links to the file. To share a file with another user, first give the user permission to read from and write to the file. You may also have to change the access permissions of the parent directory of the file to give the user read, write, or execute permission. Once the permissions are appropriately set, the user can create a link to the file so that each of you can access the file from your separate directory hierarchies. A link can also be useful to a single user with a large directory hierarchy. You can create links to cross-classify files in your directory hierarchy, using different classifications for different tasks.

### Hard Links

A hard link to a file appears as another file. If the file appears in the same directory as the linked-to file, the links must have different filenames because two files in the same directory cannot have the same name. You can create a hard link to a file only from within the filesystem that holds the file. A hard link is a direct link to the data on disk. This means data can be accessed directly via an original filename or a hard link. Both the original file and the hard link are direct links to the data on disk. The use of a hard link allows multiple filenames to be associated with the same data on disk.

### Soft Links

These are also known as symbolic links or symlink. It refers to a symbolic path indicating the abstract location of another file. A symbolic link (also sometimes known as a soft link) does not link directly to the data on disk but to another link to the data on disk. On most operating systems folders may only be linked using a symlink. Symbolic links can link across file systems to link a folder on an external hard drive.

### Hard link vs. Soft link in Linux

Hard links cannot link directories and cross file system boundaries. Soft or symbolic links are just like hard links. It allows to associate multiple filenames with a single file. However, symbolic links allows: to create links between directories and can cross file system boundaries. These links behave differently when the source of the link is moved or removed. Symbolic links are not updated. Hard links always refer to the source, even if moved or removed.

## Summary

- The oldest utility, who, produces a list of users who are logged in on the local system, the device each person is using, and the time each person logged in. The w and finger utilities show more detail, such as each user's full name and the command line each user is running.
- On systems where security is a concern, the system administrator may disable finger.
- Email enables you to communicate with users on the local system and, if the installation is part of a network, with other users on the network. If you are connected to the Internet, you can communicate electronically with users around the world.
- The popular graphical email program is sylpheed in Linux.
- On a standard Linux system, each user starts with one directory, to which the user can add subdirectories to any desired level.
- One major strength of the Linux filesystem is its ability to adapt to users' needs.
- When you refer to the tree, up is toward the root and down is away from the root.
- If you keep the filenames short, they are easy to type. The disadvantage of short filenames is that they are typically less descriptive than long filenames.

**Keywords**

- **Finger:** This utility is used to retrieve information about users on remote systems if the local system is attached to a network.
- **Filesystem:** A filesystem is a set of data structures that usually resides on part of a disk and that holds directories of files.
- **Ordinary files:** These are simply files, appear at the ends of paths that cannot support other paths.
- **Directory files:** These are also referred to as directories or folders, are the points that other paths can branch off from.
- **Pathname:** A pathname is a series of names that trace a path along branches from one file to another.
- **Startup files:** These appear in your home directory, give the shell and other programs information about you and your preferences.
- **Cd utility:** The cd (change directory) utility makes another directory the working directory but does not change the contents of the working directory.

**Self Assessment**

1. Which of these utilities is used to change to another working directory?  
A. mkdir  
B. cd  
C. mv  
D. None of the above
2. Any pathname that does not begin with \_ is a relative pathname.  
A. /  
B. ~  
C. Either / or ~  
D. None of the above
3. When you refer to the tree, \_\_\_\_ is towards the root and \_\_\_\_ is away from the root.  
A. up, down  
B. down, up  
C. left, right  
D. right, left
4. Which of these files appear at the ends of paths that cannot support other paths?  
A. Directory files  
B. Ordinary files  
C. Base files  
D. None of the above
5. Which of these builtin is used to display the pathname of the working directory?  
A. pwd  
B. work  
C. path



D. None of the above

6. With a slash (/), we represent

- A. Home directory
- B. Root directory
- C. Path directory
- D. None of the above

7. Which of these utilities is used for communication when the recipient is not logged in?

- A. echo
- B. email
- C. cat
- D. None of the above

8. The utility *who* produces the

- A. List of users who are logged in on the local system
- B. Device each person is using
- C. The time each person is logged in
- D. All of the above mentioned

9. On those systems, where security is concern, the system administrator can disable

- \_\_\_\_\_
- A. echo
  - B. finger
  - C. ls
  - D. None of the above

10. Which of these is used for establishing and ending the conversation?

- A. o
- B. oo
- C. Both of the above
- D. None of the above

11. Which of these utilities preserve ACLs?

- A. cp
- B. mv
- C. Both cp and mv
- D. None of the above

12. Which of these links exists in Linux?

- A. Hard links
- B. Soft links
- C. Water links

D. Any of these

13. Which of these files are the points that other paths can branch off from?

- A. Directory files
- B. Ordinary files
- C. Base files
- D. None of the above

14. Which of these builtin is used to display the pathname of the working directory?

- A. pwd
- B. work
- C. path
- D. None of the above

15. Which of these files appear at the ends of paths that cannot support other paths?

- A. Directory files
- B. Ordinary files
- C. Base files
- D. None of the above

### **Answers for Self Assessment**

- |       |       |       |       |       |
|-------|-------|-------|-------|-------|
| 1. B  | 2. C  | 3. A  | 4. B  | 5. A  |
| 6. B  | 7. B  | 8. D  | 9. B  | 10. C |
| 11. C | 12. C | 13. A | 14. A | 15. B |

### **Review Questions**

1. Which utilities can be used to obtain system and user information? Explain.
2. How can we communicate with other users in Linux?
3. What is a filesystem? Write the strengths of linux filesystem.
4. What are directory files and ordinary files? Explain their differences with examples.
5. What is a pathname? Write its types.
6. What is a working directory? Explain its significance. Write the utilities which are used to work with directories.
7. What are access permissions? Explain in detail.
8. What is an access control lists? Write its features. How can we set up ACLs?
9. What is a link? Explain the difference between hard links and soft links.

**Further Readings**

Mark G. Sobell, A Practical Guide to Fedora and Red Hat Enterprise Linux, Fifth Edition, Pearson Education, Inc.

**Web Links**

<https://www.redhat.com/sysadmin/linux-access-control-lists>

<https://www.geeksforgeeks.org/soft-hard-links-unixlinux/>